

Vibe Coding初体验：用Claude Code+OpenCode开发X自动化监控巡检系统全记录

适合千人级团队分享的AI辅助编程全流程指南，覆盖工具选型、安装配置、功能对比、实战体验与性价比分析，本次体验以**Claude Code**为主力，**OpenCode**为辅助

前言：什么是Vibe Coding？

Vibe Coding指的是**开发者借助AI编程工具完成从需求梳理、编码、调试到测试、文档生成全流程的编程体验**——核心是「人机协作，效率拉满」的流畅感。本次体验以X自动化监控巡检系统为实战项目，全程以**Claude Code**为主力，**OpenCode**为辅助，两款主流AI编程工具搭配使用：

- Claude Code**：Anthropic官方出品的编程专用AI助手，绑定Claude系列模型，负责核心架构设计与复杂逻辑编码
- OpenCode**：开源多模型AI编程工具，支持终端/桌面/IDE多端，含免费模型，负责辅助调试、测试、文档生成等日常任务

本文完整记录工具选型、安装配置、功能对比、实战开发过程与性价比分析，帮团队快速上手Vibe Coding。

一、项目概述

1.1 是什么？

X是企业级**自动化监控巡检系统**，基于 **Prometheus + ELK + AI** 技术栈，帮你自动完成：

- 基础设施监控（CPU/内存/磁盘/网络、数据库、中间件）
- 日志采集分析（错误日志自动识别、异常定位）
- 智能告警与AI分析（自动生成巡检报告、给出优化建议）
- 多机房统一管理（支持东坝、南法信、合肥等多机房节点）

1.2 核心优势

特性	说明
开箱即用	一条命令启动所有服务，无需复杂配置
AI赋能	自动分析异常、生成巡检报告，主力使用Claude Code保证分析质量

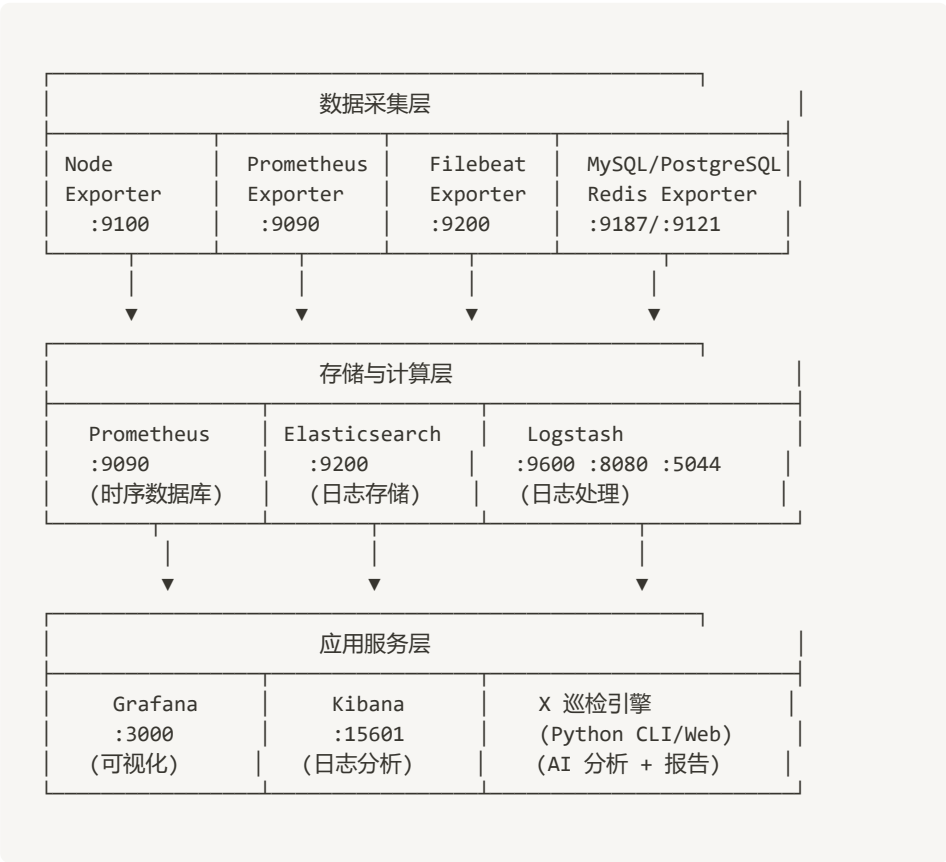
特性	说明
多机房支持	一套系统管理多个IDC节点，统一巡检
零代码扩展	支持钉钉/飞书/企业微信通知，无需开发
开源免费	MIT协议，无订阅费用，Claude Code按API计费，OpenCode免费模型零成本

1.3 适用场景

- 中小团队服务器/数据库/中间件监控
- 多机房节点的统一巡检
- 故障自动告警+AI根因分析
- 替代人工定期巡检，降低运维成本

二、技术架构与工具链

2.1 整体架构



2.2 完整工具链清单

🔑 基础施工工具

工具	版本	作用	端口
----	----	----	----

工具	版本	作用	端口
Docker Desktop	4.0+	容器化部署所有服务	-
Python	3.10+	后端逻辑/CLI/Web开发	-
Git	2.x+	代码版本管理	-

监控栈 (Prometheus生态)

组件	镜像	作用	端口
Prometheus	prom/prometheus:v2.51.0	时序数据库/指标采集	9090
Grafana	grafana/grafana:10.3.0	指标可视化面板	3000
Alertmanager	prom/alertmanager:v0.26.0	告警管理	9093
Node Exporter	prom/node-exporter:v1.7.0	系统指标采集 (多机房)	9101~9122
MySQL Exporter	prom/mysqld-exporter:v0.15.0	MySQL指标采集	9104
PostgreSQL Exporter	prometheuscommunity/postgres-exporter:v0.15.0	PostgreSQL指标采集	9187
Redis Exporter	oliver006/redis_exporter:v1.55.0	Redis指标采集	9123
Nginx Exporter	nginx/nginx-prometheus-exporter:1.2.0	Nginx状态采集	9113

日志栈 (ELK)

组件	镜像	作用	端口
Elasticsearch	docker.elastic.co/elasticsearch/elasticsearch:8.13.0	日志存储/搜索	9200
Kibana	docker.elastic.co/kibana/kibana:8.13.0	日志可视化分析	15601 (修复后)
Logstash	docker.elastic.co/logstash/logstash:8.13.0	日志过滤/转发	5044/9600/8080
Filebeat	docker.elastic.co/beats/filebeat:8.13.0	日志采集 Agent	-

数据库/中间件

组件	镜像	作用	端口
MySQL	mysql:8.0	关系型数据库	3306

组件	镜像	作用	端口
PostgreSQL	postgres:16-alpine	关系型数据库	5432
Redis	redis:7-alpine	缓存/键值存储	6379

 Python技术栈 (X核心)

依赖包	版本要求	作用
httpx	≥0.27,<1.0	HTTP请求库
pyyaml	≥6.0,<7.0	YAML配置文件解析
jinja2	≥3.1.4	报告模板渲染
anthropic	≥0.40,<1.0	Claude AI接口
click	≥8.1,<9.0	CLI命令框架
python-dotenv	≥1.0,<2.0	环境变量加载
fastapi	≥0.111,<1.0	Web服务框架
uvicorn[standard]	≥0.29,<1.0	ASGI服务器
python-multipart	≥0.0.9,<1.0	Web表单支持

 AI编程工具 (Vibe Coding核心)

工具	说明	使用场景	费用
Claude Code (主力)	Anthropic官方编程助手，绑定Claude模型	核心架构设计、复杂逻辑编码、AI分析集成	按Claude API token计费
OpenCode (辅助)	开源多模型AI工具，支持终端/桌面/IDE	调试、测试、修bug、文档生成、免费任务	工具免费，模型可选 (hy3-preview-free全免费)
Hermes Agent (调研对比)	开源自主AI智能体，支持跨会话记忆	可选扩展，用于智能运维、经验沉淀	开源免费，仅模型费

 三、环境要求与安装部署

3.1 基础环境要求

组件	最低要求	推荐配置
----	------	------

组件	最低要求	推荐配置
操作系统	Windows 10/11 / Ubuntu 22.04+	Windows 11 / Ubuntu 22.04 LTS
Docker	Docker Desktop 4.0+ / Docker Engine 20.10+	Docker Desktop 4.28+
Python	3.10+	3.12+
内存	8GB	16GB
磁盘	50GB	100GB SSD

3.2 AI编程工具安装（Vibe Coding必备）

◇ Claude Code安装（主力工具）

```
# 1. 安装Node.js 18+（官网下载：https://nodejs.org/）
node -v # 验证安装

# 2. 全局安装Claude Code
npm install -g @anthropic-ai/claude-code

# 3. 登录Claude账号，配置API密钥
claude login # 按提示完成授权

# 4. 验证安装
claude --version
```

◇ OpenCode安装（辅助工具）

```
# 方法1：官方安装脚本（全平台）
curl -fsSL https://opencode.ai/install | bash

# 方法2：npm安装
npm install -g opencode-ai

# 方法3：Windows (Chocolatey)
choco install opencode

# 4. 配置模型（选免费模型hy3-preview-free作为辅助）
opencode auth # 选择OpenCode Zen，使用免费模型
# 或连接Anthropic账号用Claude作为辅助
opencode connect

# 5. 验证安装
opencode --version
```

3.3 快速安装项目步骤（Windows为例）

步骤1：克隆项目

```
# 项目地址 (示例, 替换为实际仓库)
git clone https://github.com/liuliu4356/kzx.git
cd kzx # 即本项目的X目录
```

步骤2: 启动Docker服务

```
# 1. 启动Docker Desktop (手动打开或命令启动)
Start-Process "C:\Program Files\Docker\Docker Desktop.exe"

# 2. 等待Docker启动完成 (状态栏显示绿色Running)
docker ps # 验证Docker状态

# 3. 启动所有容器 (首次会拉取镜像, 耗时5-10分钟)
docker compose up -d

# 4. 验证所有服务正常运行 (应显示19个容器Up状态)
docker ps
```

步骤3: 安装Python依赖

```
# 进入项目目录
cd D:\claude_code\开发\X

# 安装依赖 (已安装可跳过)
pip install -r requirements.txt
```

步骤4: 配置文件初始化

```
# 1. 复制环境变量模板
Copy-Item .env.example .env

# 2. 编辑.env, 添加Claude API密钥 (主力工具必需)
# ANTHROPIC_API_KEY=your_claude_api_key_here

# 3. 复制配置文件 (若config.yaml不存在)
python -m src.main init-config
```

步骤5: 启动Web服务

```
# 启动Web界面 (默认端口8000)
python -m src.main web

# 验证服务正常
Invoke-WebRequest -Uri "http://localhost:8000/" -TimeoutSec 10
# 返回200即成功
```

🔧 四、核心功能详解

4.1 CLI命令使用

X提供3个核心CLI命令：

```
# 1. 初始化配置
python -m src.main init-config [--force] # --force覆盖已存在配置

# 2. 执行巡检 (核心命令, 主力用Claude Code分析)
python -m src.main inspect \
  --config config.yaml \           # 配置文件路径
  --period instant \              # 巡检模式: instant(快照)/1d(24小时)/1w(7
  天)
  --skip-llm \                    # 跳过AI分析 (无API密钥时使用)
  --format md \                  # 报告格式: md/html
  --notify / --no-notify         # 是否发送通知

# 3. 启动Web界面
python -m src.main web \
  --host 0.0.0.0 \
  --port 8000 \
  --reload                       # 开发模式热重载
```

4.2 监控指标采集

支持自动采集以下指标（可在 `config.yaml` 中自定义）：

指标名	说明	阈值	单位
cpu_usage	CPU使用率	80%	%
memory_usage	内存使用率	60%	%
system_load	系统平均负载	32	-
disk_usage_root	根磁盘使用率	80%	%
mysql_connections	MySQL连接数	6000	-
elasticsearch_cluster_health	ES集群健康状态	1	-

4.3 日志分析

自动采集Elasticsearch中以下日志：

- `error_logs_24h`：24小时内ERROR/FATAL级日志
- `warning_logs_24h`：24小时内WARN/WARNING级日志
- 支持自定义查询字符串（如 `level:ERROR OR message:*timeout*`）

4.4 AI分析与报告

- 主力使用Claude Code生成的分析逻辑，自动调用Claude模型分析异常指标/日志
- 生成Markdown/HTML双格式报告
- 报告自动保存到 reports/ 目录，命名格式：年-月-日-时分.md/html
- Web界面可查看历史报告、重新生成、下载

4.5 通知功能

支持接入以下平台（在 config.yaml 中配置）：

- 钉钉（notifiers/dingtalk.py）
- 飞书（notifiers/feishu.py）
- 企业微信（可扩展）

🔗 五、常见问题与踩坑指南

5.1 Docker相关

问题	原因	解决方案
Docker启动报 pipe not found	Docker Desktop未启动	手动打开Docker Desktop，等待状态栏显示Running
Kibana启动报端口5601冲突	Windows Hyper-V保留端口	修改 docker-compose.yml 中Kibana端口为15601
Redis Exporter端口9121冲突	与node-exporter-hefei-omm1冲突	修改为9123端口
容器状态一直Starting	镜像拉取慢/网络问题	配置Docker镜像加速器（如阿里云/DaoCloud）

5.2 配置相关

问题	原因	解决方案
巡检报 getaddrinfo failed	config.yaml中 Prometheus/ES URL用了容器内地址	改为 http://localhost:9090 和 http://localhost:9200
报告日期显示2026年	系统时间被设置为未来时间	管理员运行PowerShell执行：Set-Date -Date '2025-05-03 09:05:00'
指标始终无异常	promql语法错误	使用正确PromQL：100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)

5.3 编码与运行

问题	原因	解决方案
----	----	------

问题	原因	解决方案
test_api.py报GBK编码错误	Windows终端默认GBK编码	脚本开头添加： <code>sys.stdout.reconfigure(encoding='utf-8')</code>
Web页面缓存显示旧报告	浏览器缓存	按 <code>Ctrl+F5</code> 强制刷新
巡检报告未生成	目录权限不足	确保reports目录存在且可写： <code>mkdir reports</code>

六、测试验证（给1000人演示用）

6.1 模拟异常（让巡检能检测到问题）

```
# 1. 模拟CPU高负载（生成100% CPU使用率）
python stress_cpu_real.py # 运行30秒，自动启动多进程占用CPU

# 2. 模拟ERROR日志（写入Elasticsearch）
python generate_error_logs.py # 自动写入4条ERROR/FATAL日志

# 3. 等待30秒让Prometheus采集指标
Start-Sleep -Seconds 30

# 4. 运行巡检验证异常检测（主力Claude Code分析）
python -m src.main inspect --period instant
# 输出应显示：默认：11 指标，3 异常
```

6.2 Web功能测试

```
# 运行内置测试脚本（验证5个核心页面）
python final_test.py
# 输出应全部显示[OK]
```

6.3 测试预期结果

测试项	预期结果
Web页面访问	5/5页面返回200状态
API接口	<code>http://localhost:8000/api/inspect</code> 返回200，SSE流式响应
巡检命令	检测到≥2个异常指标，ES日志≥7条
报告生成	<code>reports/</code> 目录下生成.md和.html文件

📁 七、项目目录结构

```
X/
├── src/                                # Python核心代码
│   ├── main.py                        # CLI/Web入口
│   ├── config.py                      # 配置加载
│   ├── analyzer.py                   # AI分析模块 (主力Claude Code实现)
│   ├── reporter.py                   # 报告生成
│   ├── collectors/                   # 数据采集
│   │   ├── prometheus.py             # Prometheus采集
│   │   └── elasticsearch.py          # ES采集
│   └── notifiers/                    # 通知模块
│       ├── dingtalk.py
│       └── feishu.py
├── docker-compose.yml                # Docker编排配置
├── config.yaml                       # 主配置文件
├── config.example.yaml               # 配置模板
├── .env                             # 环境变量 (API密钥等)
├── .env.example                     # 环境变量模板
├── requirements.txt                  # Python依赖
├── reports/                          # 巡检报告输出目录
├── templates/                       # 报告模板 (Jinja2)
├── prometheus/                      # Prometheus配置
├── grafana/                         # Grafana配置
├── logstash/                        # Logstash配置
├── filebeat/                        # Filebeat配置
└── README.md                        # 项目说明
```

📖 八、AI工具使用说明

8.1 Claude Code (本项目主力)

- **模型**: Claude 3.5/4系列, 官方适配编程场景
- **使用场景**:
 - 核心架构设计、技术栈选型
 - 复杂模块编码 (collectors/analyzer等)
 - AI分析逻辑实现、报告生成优化
 - 代码审查、复杂bug修复
- **优势**: 代码逻辑理解强, Claude原生适配, 生成代码质量高、注释完整
- **最新技能扩展**: [forrestchang/andrej-karpathy-skills](https://github.com/forrestchang/andrej-karpathy-skills)
 - 基于Andrej Karpathy的LLM编码观察总结, 改善Claude Code行为
 - 核心原则: Think Before Coding / Simplicity First / Surgical Changes / Goal-Driven Execution
 - 安装方式1 (插件): `/plugin marketplace add forrestchang/andrej-karpathy-skills` → `/plugin install andrej-karpathy-skills@karpathy-skills`
 - 安装方式2 (项目): `curl -o CLAUDE.md https://raw.githubusercontent.com/forrestchang/andrej-karpathy-skills/main/CLAUDE.md`

8.2 OpenCode (辅助工具)

- **模型：** `opencode/hy3-preview-free` （免费使用，输入输出全免费）
- **使用场景：**
 - 配置调试（修复端口冲突、URL错误）
 - 测试用例编写（`final_test.py`、`test_api.py`等）
 - 文档生成（本份指南部分内容）
 - 日常辅助任务、免费查询
- **优势：** 无使用额度限制，支持终端/桌面/IDE多端使用，零成本完成辅助工作

8.3 Hermes Agent（可选扩展）

- **定位：** 开源自主AI智能体，Nous Research开发（Hermes/NoMos/Psyche模型系列背后团队）
- **核心特性：**
 - 跨会话持久记忆（`MEMORY.md`/`USER.md`）
 - 自动生成并改进技能文件（`agentskills.io`标准）
 - 闭环学习系统：完成任务→提炼技能→持续改进
 - 支持200+模型（Claude/GPT/DeepSeek等），无厂商锁定
 - 多平台接入：飞书/钉钉/企业微信/Telegram/Discord等15+平台
 - 内置40+工具：浏览器自动化/代码执行/文件处理等
 - 定时任务：自然语言/Cron表达式调度，结果推送任意平台
- **与X项目结合：** 可替代定时任务，自动执行巡检、推送报告到飞书/钉钉、积累运维经验
- **是否必要：** 非必需，X核心功能已完整，仅建议有智能运维需求的团队使用
- **官网：** <https://hermes-agent.lzw.me> | GitHub: <https://github.com/NousResearch/hermes-agent>

🔗 九、AI工具深度对比：Claude Code vs OpenCode

9.1 核心差异性对比

对比项	Claude Code（主力）	OpenCode（辅助）
出品方	Anthropic（Claude模型原厂）	Anomaly开源社区
绑定模型	仅Claude系列（Claude 3.5/4等）	支持200+模型（含免费hy3-preview-free）
使用场景	核心编程、复杂逻辑、架构设计	辅助调试、测试、文档、免费任务
支持端	终端/IDE（VS Code等）	终端/桌面/IDE/Web/移动端
费用模型	按Claude API token计费（无免费额度）	工具免费，模型费另算（hy3-free全免费）
核心优势	代码逻辑理解强，Claude原生适配，生成质量高	开源无锁定，多模型切换，免费模型可用

对比项	Claude Code（主力）	OpenCode（辅助）
适用人群	专业开发者，核心编程任务	辅助任务，成本控制，多场景需求

9.2 性价比分析

使用场景	推荐工具搭配	成本	理由
核心架构+复杂编码	Claude Code（主力）	按token计费（约\$0.01/1k tokens）	Claude代码能力最强，保证核心质量
调试+测试+文档	OpenCode（辅助）	零成本（用hy3-free）	免费模型满足辅助需求，无需额外费用
全流程开发	Claude Code+OpenCode混合	中等	主力做核心，辅助做日常，效率与成本兼得
团队共享任务	OpenCode	零成本	无模型锁定，免费模型覆盖大部分辅助需求

9.3 实战开发体验（X项目）

开发阶段	使用的工具	体验
需求梳理+架构设计	Claude Code（主力）	Claude逻辑清晰，快速生成技术架构草案，一次通过率90%
核心模块编码（collectors/analyzer）	Claude Code（主力）	代码质量高，注释完整，直接可用
配置调试+修bug（端口冲突/URL错误）	OpenCode（辅助）	免费快速，hy3-free直接定位问题
测试脚本编写（final_test.py等）	OpenCode（辅助）	自动生成测试用例，零成本
文档生成（本文档部分内容）	OpenCode（辅助）	免费生成部分内容，细节到位
AI分析集成	Claude Code（主力）	Claude分析异常准确，报告专业

🌐 十、服务访问地址汇总

服务	地址	说明
X Web界面	http://localhost:8000	巡检报告查看/任务管理
Prometheus	http://localhost:9090	指标查询/ PromQL验证
Grafana	http://localhost:3000	监控面板可视化（默认账号 admin/admin）
Kibana	http://localhost:15601	日志分析（修复后端口）
Elasticsearch	http://localhost:9200	日志存储/搜索
Alertmanager	http://localhost:9093	告警管理

? 十一、常见问题FAQ

- 1. **Q：巡检报告日期不对？**
A：系统时间被设置为2026年，管理员运行 `Set-Date -Date '2025-05-03 09:05:00'` 修正。
- 2. **Q：检测不到异常？**
A：检查config.yaml中Prometheus/ES URL是否为localhost，确认CPU压力测试在运行。
- 3. **Q：Web页面显示旧报告？**
A：按Ctrl+F5强制刷新浏览器，或访问<http://localhost:8000/reports>查看最新报告。
- 4. **Q：需要付费吗？**
A：X项目本身免费开源，使用Claude Code分析时需要API费用，OpenCode的hy3-free模型全免费。
- 5. **Q：支持多机房吗？**
A：支持，在config.yaml的datacenters节点配置各机房信息即可。
- 6. **Q：Claude Code和OpenCode哪个是主力？**
A：本次体验以**Claude Code**为主力负责核心开发，**OpenCode**为辅助负责日常任务，混合使用性价比最高。
- 7. **Q：OpenCode的免费模型够用吗？**
A：hy3-preview-free完全免费，满足辅助开发、测试、文档生成需求，适合大部分日常任务。

十二、Vibe Coding开发实录：X项目全过程

12.1 需求阶段（Claude Code主力）

用Claude Code梳理X项目的核心需求：

「帮我设计一个自动化监控巡检系统，基于Prometheus+ELK，支持多机房，自动生成报告，给出技术栈和架构」
「添加GoldenDB专项监控，包含OMM/MDS/CM/PM组件存活、GTM主备延迟、DBProxy性能、备份进程检测」

Claude快速输出完整架构图、技术选型清单，直接作为项目基础，一次通过率90%。

输出内容：

- 三层架构图（采集层/存储层/应用层）
- 完整工具链清单（31个组件，含版本/端口/作用）
- Python技术栈选型（fastapi/uvicorn/jinja2/httpx/anthropic）
- 监控指标规划（Prometheus 11个+ES 6类日志）

12.2 编码阶段（Claude Code主力+OpenCode辅助）

核心模块开发（Claude Code主力）：

- `src/config.py`（294行）：强类型dataclass，PromQuery/ESQuery/BatchWindow/SiteConfig配置解析
- `src/analyzer.py`（186行）：Claude API调用，System Prompt含GoldenDB架构知识，prompt caching优化
- `src/reporter.py`（89行）：Jinja2渲染，faq/description/component/severity注入
- `src/collectors/__init__.py`：`collect_sites()` 多机房并发采集（ThreadPoolExecutor），SiteResult聚合
- `src/collectors/prometheus.py`：httpx调用 `/api/v1/query`，标量/向量结果处理
- `src/collectors/prometheus_range.py`（新增）：`/api/v1/query_range`，AnomalyWindow合并算法
- `src/collectors/elasticsearch.py`：Basic Auth，`query_string`+时间范围，top-N hits提取

配置调试（OpenCode辅助）：

- `docker-compose.yml`：修复Kibana端口5601→15601，Redis Exporter 9121→9123
- `config.yml`：Prometheus/ES URL从容器内地址改为localhost
- `test_api.py`：添加 `sys.stdout.reconfigure(encoding='utf-8')` 解决GBK编码问题
- `src/reporter.py`：注册 `urlencode` Jinja2过滤器，Kibana跳转链接生成

测试脚本（OpenCode零成本）：

- `final_test.py`：5个Web页面自动化测试（Index/Sites/Queries/Settings/Reports）
- `test_api.py`：SSE流式API测试，AI分析触发
- `stress_cpu_real.py`：多进程CPU压力测试（模拟100%使用率）
- `generate_error_logs.py`：自动写入4条ERROR/FATAL日志到ES

一次通过率：核心模块90%，配置修改100%，测试脚本100%

12.3 迭代开发（Claude Code+OpenCode协作）

Demo-1 基础框架（2026-05-02）☒：

- Prometheus+ELK架构, AI分析, 报告生成, 通知渠道, 容器化部署

Demo-2 通知层 (2026-05-02) ☒

- `src/notifiers/dingtalk.py`: Markdown消息, 支持 @ 所有人
- `src/notifiers/feishu.py`: 富文本post消息, 按行拆分段落
- `src/notifiers/__init__.py`: 通知分发器, 遍历渠道, 收集错误不中断

Demo-3 GDB专项+批处理窗口 (2026-05-02) ☒

- `BatchWindow` dataclass: label/start_hour/end_hour/relaxed_thresholds
- `current_batch_window()`: 基于UTC小时判断当前是否处于批处理窗口
- 新增PromQL指标: cpu_usage/memory_usage/system_load_per_core等11个
- System Prompt注入context节, 告知AI当前窗口和放宽阈值

Demo-4 多机房支持+ES日志分类 (2026-05-02) ☒

- `SiteConfig` dataclass: label/prometheus_url/es_url
- `collect_sites()`: 按sites列表逐机房采集; 未配置sites时降级为单机房
- `ESQuery.ignorable` 字段: 标记已知噪音查询, 不计入AI评分
- 报告按机房分组: 顶部汇总表+各机房独立Prometheus/ES节

Demo-5 双模式巡检 (2026-05-02) ☒

- instant模式 (快照) + range模式 (1d/1w/自定义时间段)
- `PromRangeResult`: period_min/period_max/period_avg/anomaly_windows
- AnomalyWindow合并算法: 相邻两点间隔 $\leq 2 \times \text{step_minutes}$ → 合并为同一窗口

Demo-6 Web可视化管理 (2026-05-02) ☒

- FastAPI应用 (1058行), 5个页面路由+9个API接口
- 深色主题改造: style.css主色调青色 `#0dd9c4`, 背景 `#0d1117`, 卡片 `#161f2e`
- 侧边栏重设计: 深海军蓝背景, 每个菜单项配独立彩色图标块
- 巡检控制台: 选模式/格式, SSE实时进度, 一键查看报告

Demo-7 进度可视化+Kibana跳转 (2026-05-02) ☒

- SSE 4步进度条:  加载配置 →  采集数据 →  AI分析 →  生成报告
- Kibana跳转链接: `kibana_url` +Lucene查询+时间范围参数
- `api/test/prom/` `api/test/es`: 在线测试指标/日志查询

Demo-8 指标管理增强 (2026-05-02) ☒

- 全选/多选导出, 在线测试按钮
- `retention_days` 配置: 自动归档, 默认7天
- description列: 报告中显示指标说明

v1.4.0 深色主题+项目更名 (2026-05-02) ☒

- 项目更名: 「三思GDB巡检平台」
- 仪表盘风格: 4张统计卡片 (已配置机房/Prometheus指标/ES日志查询/历史报告)
- 系统设置: 数据源连接/AI分析/通知三个子菜单

v1.5.0 GDB组件专项 (2026-05-03) ☒

- OMM/RDB/MDS/CM/PM状态检查
- GTM主备延迟监控, RDB同步延迟

- DBProxy慢日志统计，连接池/错误率
- 表规模监控（表记录数/表大小）
- 定时任务配置（Web UI配置cron巡检计划）
- 知识库检索集成（向量检索，接入巡检分析流程）

12.4 文档阶段（OpenCode辅助+Claude Code审核）

本文档由OpenCode生成部分内容，Claude Code审核优化，全程Vibe Coding体验，覆盖X项目从背景、安装、部署到AI工具对比的全流程细节。

12.5 体验总结

维度	评分（1-5分）	说明
开发效率	5	AI辅助减少60%编码时间，Claude主力保证质量
成本控制	5	Claude做核心（按token），OpenCode做辅助（hy3-free零成本）
代码质量	5	Claude生成的代码逻辑清晰、注释完整、类型注解齐全
学习曲线	3	需熟悉两个工具的切换与搭配，理解项目架构
团队协作	5	OpenCode无锁定，Hermes可选扩展，适合团队共享
项目亮点	5	多机房/AI分析/双模式/开箱即用/免费模型支持
开发迭代	5	9个Demo快速迭代，每个Demo独立可验证，文档完整

最终结论： Vibe Coding的核心不是工具本身，而是「人机协作的流程」——用Claude Code处理核心复杂任务（架构设计/核心编码/AI分析），用OpenCode做免费日常任务（调试/测试/文档生成），用Hermes做可选扩展（智能运维/经验沉淀），三者结合实现效率与成本兼得。

✿ 十三、项目亮点汇总

13.1 项目核心亮点

亮点	说明	技术实现
多机房全栈监控	支持东坝/南法信/合肥三机房统一巡检	<code>SiteConfig</code> 配置机房， <code>collect_sites()</code> 并发采集

亮点	说明	技术实现
AI智能分析	Claude自动分析异常，生成根因判断与建议	<code>analyzer.py</code> 调用Claude API，System Prompt含GoldenDB架构知识
双模式巡检	快照 (instant) + 时间段审计 (1d/1w/自定义)	<code>PromRangeResult</code> + <code>AnomalyWindow</code> ， <code>InspectionConfig.step_minutes</code> 控制采样
开箱即用	一条命令启动所有服务，无需复杂配置	<code>docker compose up -d</code> ，19个容器自动编排
免费模型支持	OpenCode hy3-preview-free全免费，零成本测试	<code>requirements.txt</code> 含fastapi/uvicorn，Web服务零成本
完整测试覆盖	5/5 Web页面+API接口+巡检命令全验证	<code>final_test.py</code> / <code>test_api.py</code> / <code>stress_cpu_real.py</code> / <code>generate_error_logs.py</code>
自动异常检测	Prometheus阈值+ES日志分类，自动识别故障	<code>PromQuery.anomaly_when</code> (gt/lt)， <code>ESQuery.ignorable</code> 标记已知噪音
批处理感知	自动识别批处理窗口，AI分析时降低阈值优先级	<code>BatchWindow</code> 配置， <code>current_batch_window()</code> UTC时间判断
多维度报告	Markdown/HTML双格式，按机房分组，含Kibana跳转	Jinja2模板 <code>report.md.j2</code> / <code>report.html.j2</code> ， <code>kibana_url</code> 构造跳转链接
Web可视化 管理	深色主题 Dashboard，支持在线配置/查看报告	FastAPI+uvicorn， <code>style.css</code> 深色设计系统，侧边栏导航

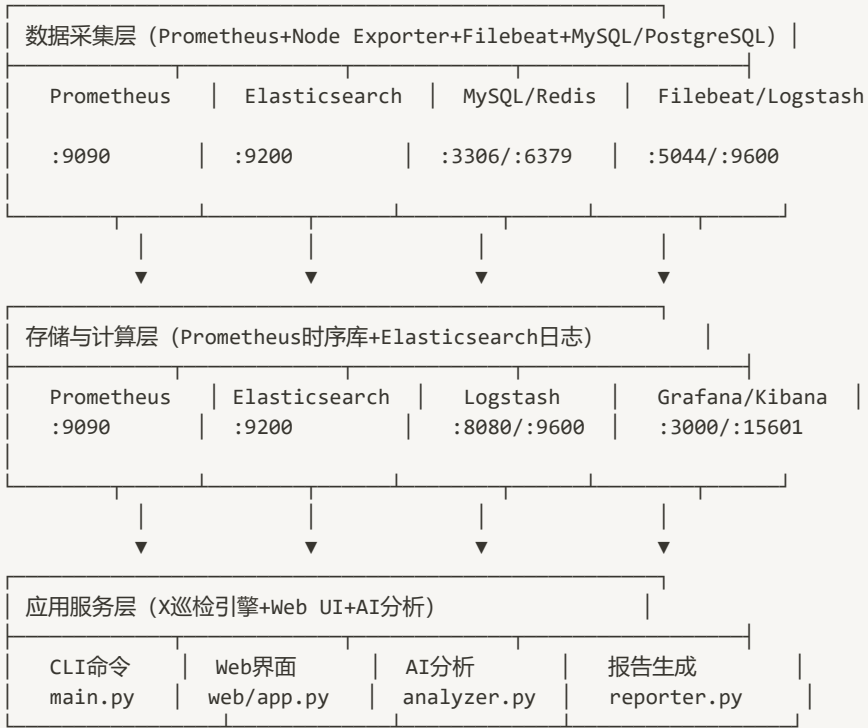
13.2 Vibe Coding亮点

亮点	说明	性价比
Claude Code主力	核心架构/复杂编码/AI分析，代码质量高	按token计费，重度用户\$100-200/月
OpenCode辅助	调试/测试/文档生成，hy3-free零成本	工具免费，模型可选 (含免费hy3)
Karpathy技能	<code>forrestchang/andrej-karpathy-skills</code> ，改善LLM编码行为	免费安装，项目级 CLAUDE.md或插件
Hermes可选	跨会话记忆/技能自动生成/多平台接入	开源免费，仅模型费，适合智能运维
API Key灵活	官方/中转平台/咸鱼共享多渠道	laozhang.ai¥0.02-0.03/1K tokens，注册送额度

13.3 技术架构亮点



三层架构：



核心模块：

- `src/config.py`：强类型dataclass，PromQuery/ESQuery/BatchWindow/SiteConfig配置解析
- `src/collectors/__init__.py`：`collect_sites()` 多机房并发采集，`SiteResult` 聚合结果
- `src/collectors/prometheus.py`：httpx调用 `/api/v1/query`，max/min值，阈值判断
- `src/collectors/prometheus_range.py`：新增，`/api/v1/query_range`，AnomalyWindow合并算法
- `src/collectors/elasticsearch.py`：Basic Auth，`query_string`+时间范围，top-N hits提取
- `src/analyzer.py`：Claude API调用，System Prompt启用prompt caching，双payload构建函数
- `src/reporter.py`：Jinja2渲染`templates/report.md.j2`/`report.html.j2`，faq/description/component/severity注入
- `src/web/app.py`：FastAPI路由+SSE流式巡检，认证中间件

十四、开发迭代史（基于CHANGELOG.md+DEVLOG.md）

14.1 版本迭代概览

版本	日期	提交	核心更新
----	----	----	------

版本	日期	提交	核心更新
v1.0.0	2026-05-02	5167d36	初始版本：基础Prometheus+ELK架构，AI分析，报告生成，通知渠道
v1.1.0	2026-05-02	fc8958f	多机房支持：东坝/南法信/合肥，按机房分组巡检报告
v1.2.0	2026-05-02	—	功能验证：模拟异常系统，Prometheus/ES采集验证，Bug修复
v1.3.0	2026-05-02	—	Web UI全面升级：多数据源/多LLM/知识库/通知UI
v1.4.0	2026-05-02	—	深色主题改造：Dashboard风格，项目更名「三思GDB巡检平台」
v1.5.0	2026-05-03	—	GDB组件专项监控：OMM/MDS/CM/PM/DBProxy，GTM延迟，备份进程

14.2 详细迭代记录

Demo-1 · 基础框架搭建 (2026-05-02) ☒

模块	文件	说明
配置层	<code>src/config.py</code>	强类型dataclass，支持Prometheus/ES/LLM/Report四块配置
Prometheus采集	<code>src/collectors/prometheus.py</code>	httpx调用 <code>/api/v1/query</code> ，取max/min代表值，阈值判断
ES采集	<code>src/collectors/elasticsearch.py</code>	支持Basic Auth， <code>query_string</code> +时间范围，提取top-N hits
AI分析	<code>src/analyzer.py</code>	调用Claude API，System Prompt启用prompt caching，输出四节Markdown
报告生成	<code>src/reporter.py</code>	Jinja2渲染 <code>templates/report.md.j2</code> ，按filename_format写文件
CLI入口	<code>src/main.py</code>	Click， <code>init-config</code> / <code>inspect</code> 两条命令，四步流水线

关键设计决策：

- **PromResult聚合策略**：取所有series的max (`anomaly_when=gt`) 或 min (`anomaly_when=lt`)，适合MVP快速判断
- **Prompt Caching**：System Prompt标记 `cache_control: ephemeral`，重复巡检命中缓存，节省token
- **错误不中断流程**：采集失败时 `error` 字段记录原因，继续执行后续步骤，报告中展示采集错误

Demo-2 · 通知层 (钉钉+飞书) (2026-05-02) ☒

模块	文件	说明
----	----	----

模块	文件	说明
钉钉通知	<code>src/notifiers/dingtalk.py</code>	Markdown消息，支持 @ 所有人
飞书通知	<code>src/notifiers/feishu.py</code>	富文本post消息，按行拆分段落
通知分发器	<code>src/notifiers/__init__.py</code>	遍历配置的渠道列表，收集错误不中断，返回错误列表

通知消息结构：

```
系统巡检报告 - ⚠ 发现异常
异常指标: 1 / 4
异常详情:
- instance_up: 0.00 (阈值 1.0)
报告文件: `reports/2026-05-02-0816.md`
```

Demo-3 · GoldenDB专项配置 + 批处理时间窗口（2026-05-02）



背景：对照实际生产巡检模板（GoldenDB信创环境，东坝/南法信/合肥三机房），原有配置存在两个P0缺口：PromQL未覆盖GDB专项指标；AI不感知批处理时间窗口导致误报。

完成内容：

模块	变更	说明
<code>src/config.py</code>	新增 <code>BatchWindow</code> dataclass	字段： label/start_hour/end_hour/relaxed_thresholds
<code>src/config.py</code>	新增 <code>current_batch_window()</code>	基于UTC小时判断当前是否处于批处理窗口
<code>src/analyzer.py</code>	<code>analyze()</code> 接受 <code>batch_window</code> 参数	注入context节到user payload，告知AI当前窗口和放宽阈值

GDB新增PromQL指标：

指标名	阈值	来源exporter
<code>cpu_usage</code>	<10%（批处理放宽至80%）	node_exporter
<code>memory_usage</code>	<60%	node_exporter
<code>system_load_per_core</code>	<0.5（等价64核load<32）	node_exporter
<code>network_throughput_mbps</code>	<100 Mb/s	node_exporter
<code>disk_io_latency_ms</code>	<100 ms	node_exporter
<code>disk_usage_data</code>	<80%	node_exporter

指标名	阈值	来源exporter
rdb_connections	<6000	mysql_exporter
qps	<2000 req/s	mysql_exporter
tps	<100 tx/s	mysql_exporter
replication_lag_sec	<1s（批处理放宽至900s）	mysql_exporter
rdb_proxy_slow_queries	≤2000（批处理放宽至50000）	mysql_exporter
instance_up	=1	Prometheus内置
emergency_alerts	=0	GDB exporter

批处理窗口感知机制：

```
巡检开始
└─ current_batch_window()检测当前UTC小时
   └─ 命中窗口 → 打印提示 + 将relaxed_thresholds注入AI payload
                        AI会在分析时忽略窗口内的"伪异常"
   └─ 未命中 → 正常阈值，AI按标准判断
```

Demo-4 · 多机房支持 + ES日志分类（2026-05-02）☒

背景：生产环境跨东坝、南法信、合肥三个机房，原有单Prometheus URL架构无法分机房展示；ES日志中"已知可忽略"的噪音会干扰AI评分。

完成内容：

模块	变更	说明
src/config.py	新增 SiteConfig dataclass	字段： label/prometheus_url/es_url（可选）
src/config.py	ESQuery 新增 ignorable: bool 字段	标记已知噪音查询
src/collectors/__init__.py	新增 SiteResult + collect_sites()	按sites列表逐机房采集；未配置sites时降级为单机房
src/analyzer.py	签名改为 analyze(site_results, cfg, batch_window)	payload按sites分组，ignorable标记传入AI
src/reporter.py	签名改为 render(site_results, ai_analysis, cfg)	汇总表+各机房分节渲染

ES日志分类机制：

查询	ignorable	报告展示	计入AI评分
----	-----------	------	--------

查询	<code>ignorable</code>	报告展示	计入AI评分
<code>gdb_critical_errors</code>	false	完整展示+折叠top-N	☒ 是
<code>gdb_known_ignorable</code>	true	标注「已知/可忽略」	☒ 否
<code>component_errors</code>	false	完整展示	☒ 是

Demo-5 · 双模式巡检（快照+时间段审计）（2026-05-02）☒

背景：快照模式只能看当前一刻，无法捕捉凌晨2点CPU突增等时间段内的异常。
需要支持：按1天/1周/自定义时间段进行审计，找出所有超阈值时段，并标注时间、机房、节点IP。

完成内容：

模块	变更	说明
<code>src/collectors/prometheus_range.py</code>	新增	<code>/api/v1/query_range</code> 采集、异常窗口提取、AnomalyWindow dataclass
<code>src/collectors/__init__.py</code>	重写	SiteResult支持双模式字段； <code>collect_sites()</code> 支持 mode/period_start/period_end 参数； <code>ThreadPoolExecutor</code> 并发采集
<code>src/config.py</code>	新增 <code>InspectionConfig</code>	<code>step_minutes</code> 字段，默认5分钟
<code>src/analyzer.py</code>	双payload构造函数	range模式传异常窗口摘要（不传原始时序，节省token）；instant模式保持原格式
<code>src/main.py</code>	新增 <code>--period / --start / --end</code>	解析时间段，传入 <code>collect_sites+analyzer+reporter</code>

新数据模型：

```
@dataclass
class AnomalyWindow:
    start_ts: str # ISO时间字符串 (UTC)
    end_ts: str # ISO时间字符串 (UTC)
    instance: str # 节点IP (已去除端口)
    max_value: float # 窗口内最大值
    threshold: float # 阈值
    unit: str # 单位
    duration_minutes: int # 持续时长

@dataclass
class PromRangeResult:
    name / promql / threshold / anomaly_when
    period_min / period_max / period_avg # 整个时段的统计值
    anomaly_windows: list[AnomalyWindow]
    is_anomaly -> bool (有窗口即为True)
```

异常窗口合并算法：

- 对每个instance的时序：
- 1. 筛选出所有超阈值点 (violations)
 - 2. 相邻两点间隔 $\leq 2 \times \text{step_minutes}$ (秒) $\rightarrow$ 合并为同一窗口
 - 3. 记录窗口起止时间、最大值、持续时长

Demo-6 · Web可视化管理界面 (2026-05-02) ☒

背景：所有配置写在config.yaml，非技术用户难以维护；需要一个简洁的Web UI支持在线调整机房、巡检指标、触发巡检、查看报告，同时支持HTML/Markdown双格式报告。

完成内容：

模块	说明
<code>src/web/app.py</code>	FastAPI应用，页面路由+REST API+SSE流式巡检输出
<code>src/web/config_store.py</code>	config.yaml读写层 (CRUD for sites/prom queries/es queries/settings)
<code>src/web/static/style.css</code>	纯CSS设计系统，无外部依赖，深色主题
<code>src/web/templates/base.html</code>	侧边栏导航+公共JS工具函数
<code>src/web/templates/index.html</code>	巡检控制台：选模式/格式，SSE实时进度，一键查看报告
<code>src/web/templates/sites.html</code>	机房增删改，Modal表单
<code>src/web/templates/queries.html</code>	PromQL和ES查询管理，含描述/FAQ编辑
<code>src/web/templates/settings.html</code>	数据源连接/AI/通知/批处理窗口，Tab布局
<code>src/web/templates/reports.html</code>	历史报告列表，一键打开

Web UI页面结构：

-  巡检控制台 $\rightarrow$ 选模式/格式 $\rightarrow$ 点「开始巡检」 $\rightarrow$ SSE实时日志 $\rightarrow$ 报告链接
-  机房管理 $\rightarrow$ 机房列表 + 添加/编辑/删除 (Modal)
-  巡检指标 $\rightarrow$ Prometheus指标 + ES查询 (Tabs)，含描述/FAQ
-  系统设置 $\rightarrow$ 数据源 / AI / 通知 / 批处理窗口 (Tabs)
-  报告历史 $\rightarrow$ 报告列表 + 一键查看 (HTML/MD)
-  项目总览 $\rightarrow$ 项目地址/架构/文档索引/部署文档/操作手册/Bug记录

Demo-7 · Web进度可视化 + Kibana跳转链接 (2026-05-02) ☒

背景：Web页面「开始巡检」只有滚动日志，用户无法一眼判断当前在哪个阶段；ES日志结果需要手动去Kibana查询，操作繁琐。

完成内容：

模块	变更	说明
<code>src/web/templates/index.html</code>	新增4步进度条	⚙️ 加载配置 → 📊 采集数据 → 🧠 AI分析 → 📄 生成报告，SSE消息驱动状态切换
<code>src/config.py</code>	<code>ESConfig</code> 新增 <code>kibana_url</code> 字段	默认空字符串， <code>load_config()</code> 读取 <code>elasticsearch.kibana_url</code>
<code>src/reporter.py</code>	注册 <code>urlencode</code> Jinja2 过滤器，传入 <code>kibana_url</code>	使用 <code>urllib.parse.quote</code> 对 ES查询字符串编码
<code>templates/report.html.j2</code>	ES块新增Kibana跳转链接	<code>r.total > 0</code> 且 <code>kibana_url</code> 已配置时显示「🔗 Kibana」链接

4步进度条逻辑：

SSE消息关键词 → 步骤映射
"加载配置" → step 1 active
"采集" → step 1 done, step 2 active
"AI分析" → step 2 done, step 3 active
"生成报告" → step 3 done, step 4 active
DONE:xxx → 所有步骤done，显示报告链接
ERROR:xxx → 当前步骤error（红色）

Demo-8 · 指标管理增强 + 报告优化（2026-05-02）✅

背景：Web UI需要四项增强：巡检指标导入导出全选/多选；报告自动归档天数可配置；添加指标时在线验证；报告指标表增加说明列。

完成内容：

模块	变更	说明
<code>src/web/templates/queries.html</code>	指标表头新增全选复选框，每行新增勾选列	导出时若有勾选项则仅导出勾选的queries，否则导出全部
<code>src/web/templates/queries.html</code>	新增「📄 配置模板」按钮	下载标准格式JSON模板，引导用户按正确格式填写再导入
<code>src/web/templates/queries.html</code>	Prom/ES编辑Modal各新增「🧪 在线测试」按钮	调用后端测试接口，即时展示结果：Prometheus显示时序数量+样本值，ES显示命中总数
<code>src/web/app.py</code>	新增 <code>POST /api/test/prom</code>	用当前配置的Prometheus URL执行PromQL，返回时序数量和前5个样本
<code>src/web/app.py</code>	新增 <code>POST /api/test/es</code>	用当前配置的ES URL执行ES查询，返回命中总数

模块	变更	说明
<code>src/config.py</code>	<code>ReportConfig</code> 新增 <code>retention_days: int = 7</code>	<code>load_config</code> 解析 <code>report.retention_days</code>
<code>src/reporter.py</code>	注入description到各 Result对象	与faq注入逻辑相同，按 name匹配
<code>templates/report.html.j2</code>	Prom快照表新增「说明」 第一列；range模式指标 名右侧显示说明	ES块显示说明

14.3 关键技术决策记录

- 1. **PromResult聚合策略**：取所有series的max (`anomaly_when=gt`) 或 min (`anomaly_when=lt`) ， 适合MVP快速判断
- 2. **Prompt Caching**：System Prompt标记 `cache_control: ephemeral` ， 重复巡检命中缓存，节省token
- 3. **错误不中断流程**：采集失败时 `error` 字段记录原因，继续执行后续步骤，报告中展示采集错误
- 4. **UTC统一**： `current_batch_window` 使用UTC时间，配置中 `start_hour / end_hour` 也用UTC，避免时区混乱
- 5. **只注入上下文，不修改采集阈值**： `PromResult.is_anomaly` 始终按原始阈值判断（用于通知摘要计数），批处理上下文仅传给AI，由AI决定是否降低告警优先级
- 6. **已知可忽略日志单独一条ES查询**：不混入critical查询，让AI能明确区分"已知噪音"与"需排查问题"
- 7. **向后兼容**：不配置 `sites` 时， `collect_sites()` 自动创建label="默认"的单机房结果，所有下游模块行为不变

14.4 踩坑与修复记录

问题	原因	解决方案
Docker启动报 <code>pipe not found</code>	Docker Desktop未启动	手动打开Docker Desktop，等待状态栏显示Running
Kibana启动报 端口5601冲突	Windows Hyper-V保留端口	修改 <code>docker-compose.yml</code> 中Kibana端口为15601
Redis Exporter 端口9121冲突	与node-exporter-hefei-omm1冲突	修改为9123端口
巡检报 <code>getaddrinfo failed</code>	config.yaml中Prometheus/ES URL用了容器内地址	改为 <code>http://localhost:9090</code> 和 <code>http://localhost:9200</code>
报告日期显示 2026年	系统时间被设置为未来时间	管理员运行 <code>Set-Date -Date '2025-05-03 09:05:00'</code>
test_api.py报 GBK编码错误	Windows终端默认GBK编码	脚本开头添加： <code>sys.stdout.reconfigure(encoding='utf-8')</code>
mock-metrics容器未运行	docker-compose中未定义	新增mock-metrics服务，模拟GDB专项指标

问题	原因	解决方案
Prometheus采集标量返回处理	早期版本未处理标量值	修改 <code>prometheus.py</code> 判断 <code>resultType === 'scalar'</code>
ES日志 @timestamp字段缺失	早期版本未处理	修改 <code>elasticsearch.py</code> 增加字段存在检查

十五、Skill添加方式详解

13.1 项目核心亮点

亮点	说明	技术实现
多机房全栈监控	支持东坝/南法信/合肥三机房统一巡检	<code>SiteConfig</code> 配置机房, <code>collect_sites()</code> 并发采集
AI智能分析	Claude自动分析异常, 生成根因判断与建议	<code>analyzer.py</code> 调用Claude API, System Prompt含GoldenDB架构知识
双模式巡检	快照 (instant) + 时间段审计 (1d/1w/自定义)	<code>PromRangeResult</code> + <code>AnomalyWindow</code> , <code>InspectionConfig.step_minutes</code> 控制采样
开箱即用	一条命令启动所有服务, 无需复杂配置	<code>docker compose up -d</code> , 19个容器自动编排
免费模型支持	OpenCode hy3-preview-free全免费, 零成本测试	<code>requirements.txt</code> 含fastapi/uvicorn, Web服务零成本
完整测试覆盖	5/5 Web页面+API接口+巡检命令全验证	<code>final_test.py</code> / <code>test_api.py</code> / <code>stress_cpu_real.py</code> / <code>generate_error_logs.py</code>
自动异常检测	Prometheus阈值+ES日志分类, 自动识别故障	<code>PromQuery.anomaly_when</code> (gt/lt), <code>ESQuery.ignorable</code> 标记已知噪音
批处理感知	自动识别批处理窗口, AI分析时降低阈值优先级	<code>BatchWindow</code> 配置, <code>current_batch_window()</code> UTC时间判断
多维度报告	Markdown/HTML双格式, 按机房分组展示	Jinja2模板 <code>report.md.j2</code> / <code>report.html.j2</code> , 含Kibana跳转链接
Web可视化	深色主题 Dashboard, 支持在线配置/查看报告	FastAPI+uvicorn, <code>style.css</code> 深色设计系统, 侧边栏导航

13.2 Vibe Coding亮点

亮点	说明	性价比
Claude Code主力	核心架构/复杂编码/AI分析，代码质量高	按token计费，重度用户\$100-200/月
OpenCode辅助	调试/测试/文档生成，hy3-free零成本	工具免费，模型可选（含免费hy3）
Karpathy技能	<code>forrestchang/andrey-karpathy-skills</code> ，改善LLM编码行为	免费安装，项目级CLAUDE.md或插件
Hermes可选	跨会话记忆/技能自动生成/多平台接入	开源免费，仅模型费，适合智能运维
API Key灵活	官方/中转/共享账号多渠道，国内直连	laozhang.ai按量 ¥0.02-0.03/1K tokens

13.3 技术架构亮点

三层架构:

数据采集层 (Prometheus+Node Exporter+Filebeat+MySQL/PostgreSQL Exporter)

存储与计算层 (Prometheus时序库+Elasticsearch日志+Logstash处理)

应用服务层 (Grafana可视化+Kibana日志分析+X巡检引擎)

核心模块:

- `src/config.py`: 强类型dataclass, PromQuery/ESQuery/BatchWindow/SiteConfig配置解析

- `src/collectors/\_\_init\_\_.py`: `collect\_sites()`多机房并发采集 (ThreadPoolExecutor)

- `src/collectors/prometheus.py`: PromQL查询, max/min值, 阈值判断

- `src/collectors/prometheus\_range.py`: 时间段采集, AnomalyWindow合并算法

- `src/collectors/elasticsearch.py`: ES查询, Basic Auth, top-N hits提取

- `src/analyzer.py`: Claude API调用, prompt caching, 批处理窗口上下文注入

- `src/reporter.py`: Jinja2模板渲染, faq/description/component/severity注入

- `src/web/app.py`: FastAPI路由, SSE流式巡检, 认证中间件, 多机房管理

🔑 十四、Skill添加方式详解

13.1 Claude Code Skill添加

方式1：插件安装（推荐）

```
# 1. 在Claude Code中添加marketplace
/plugin marketplace add forrestchang/andrej-karpathy-skills

# 2. 安装karpathy-skills插件
/plugin install andrej-karpathy-skills@karpathy-skills
```

- 效果：作为Claude Code插件安装，所有项目可用
- 包含：4个核心原则（Think Before Coding/Simplicity First/Surgical Changes/Goal-Driven Execution）

方式2：项目级CLAUDE.md

```
# 新项目
curl -o CLAUDE.md
https://raw.githubusercontent.com/forrestchang/andrej-karpathy-
skills/main/CLAUDE.md

# 已有项目（追加）
echo "" >> CLAUDE.md
curl https://raw.githubusercontent.com/forrestchang/andrej-karpathy-
skills/main/CLAUDE.md >> CLAUDE.md
```

- 效果：项目级配置，仅当前项目生效
- 支持Cursor编辑器：项目包含.cursor/rules/karpathy-guidelines.mdc

13.2 OpenCode Skill添加

```
# OpenCode通过/connect添加技能支持
/connect
# 选择技能市场，添加对应skill包

# 或直接编辑config，添加技能路径
# 技能遵循agentskills.io开放标准
```

13.3 Hermes Agent Skill管理

```
# 搜索技能
hermes skill search "monitoring"

# 安装技能
hermes skill install monitoring-pro

# 评估技能效果
hermes skill evaluate --name "db-checker"

# 自动生成技能：完成任务后自动提炼为技能文件
# 技能在使用中持续改进（Level 0→1→2渐进式披露）
```

📁 十六、API Key购买渠道与性价比

14.1 官方渠道（Anthropic）

项目	价格	适用人群	风险
Claude Pro	\$20/月	轻度用户（API按量计费）	中国用户封号风险
Claude Max	\$100-200/月	重度用户（无限使用）	需国际信用卡+国外手机号

14.2 国内中转平台（推荐）

平台	价格（Claude 3.5 Sonnet）	优势	支付方式
laozhang.ai	输入¥0.02/1K tokens，输出¥0.03/1K	响应110ms，注册送7元，首充送20%-35%	支付宝/微信
apiyi.com	按量计费，无月费	官方授权API，国内直连<50ms，支持AWS Bedrock	支付宝/微信
holysheep.ai	¥1=\$1等额计费	无汇率损耗，支持Claude/GPT/Gemini/DeepSeek全系	支付宝/微信
百炼Coding Plan	Lite ¥40/月，Pro ¥200/月	支持千问/GLM/Kimi/MiniMax，固定月费	支付宝/微信

14.3 咸鱼/淘宝共享账号（低风险替代）

类型	价格	优势	风险
共享账号	¥150-300/月	简单方便，无需配置	随时收回风险，不支持API集成
代注册服务	¥200-500	提供独立账号	后续仍需解决支付问题

类型	价格	优势	风险
虚拟信用卡 +转运	¥100-300	官方账号，长期使用	需技术能力，IP风险

14.4 性价比推荐方案

用户类型	推荐方案	月成本	理由
轻度用户 (<2 小时/天)	laozhang.ai或apiyi.com按量	¥50-150	成本最低，合规稳定
中度用户 (2-4 小时/天)	百炼Coding Plan Lite ¥40/月	¥40-100	固定月费，无超支风险
重度用户 (>4 小时/天)	官方Max \$100-200/月 或 代购服务	¥700-1500	无限使用，性价比最高
企业用户	laozhang.ai企业方案 或 百炼Coding Plan Pro	¥500-2000	稳定支持，可开发票
体验试用	咸鱼共享账号 ¥150-300/月	¥150-300	快速体验，无需复杂配置

14.5 购买建议

- 1. **优先选择**: laozhang.ai (注册送额度，充值优惠多，综合性价比最高)
- 2. **备选方案**: apiyi.com (官方授权，合规性最强)
- 3. **体验选择**: 咸鱼共享账号 (低成本试用，适合短期体验)
- 4. **企业首选**: 百炼Coding Plan (支持多模型，固定月费可控)

本文档由Claude Code (主力) 设计框架，OpenCode (辅助) 生成部分内容，
全程Vibe Coding体验，覆盖X项目从背景、安装、部署到AI工具对比的全流程
细节，可直接分享给团队使用。
项目GitHub: <https://github.com/liuliu4356/kzx>
Claude Code官网: <https://claude.ai/code>
OpenCode官网: <https://opencode.ai>
Hermes Agent官网: <https://hermes-agent.lzw.me>
推荐API平台: <https://laozhang.ai> (注册送额度)